

UCS617

MICROPROCESSOR – BASED SYSTEM DESIGN

LAB ASSIGNMENT – 1 (8086 MICROPROCESSOR)

SUBMITTED TO:

DR. ROHAN SHARMA

SUBMITTED BY:

AKASH PACHAURI (102303105)

FARHAN KAZI (102303994)

JOBAN KHUSHNOOR (102303373)

BHAWESH WAR (102313065)

AGAMJOT SINGH (102303387)

SUBGROUP:

3C31



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

INDEX

S.NO	TITLE	PAGE NO
1.	Write an assembly language program to add two 16-bit numbers in 8086	1
2.	Write an assembly language program to subtract two 16-bit numbers in 8086.	3
3.	Write an assembly language program to multiply two 16-bit numbers in 8086.	5
4.	Write an assembly language program to divide two 16-bit numbers in 8086.	7
5.	Write an assembly language program to demonstrate AAA, AAS, AAM, AAD, DAA and DAS in 8086.	9
6.	Write an assembly language program to find out the count of positive numbers and negative numbers from a series of signed numbers in 8086.	15
7.	Write an assembly language program to convert to find out the largest number from a given unordered array of 8-bit numbers, stored in the locations starting from a known address in 8086.	17
8.	Write an assembly language program to convert to find out the largest number from a given unordered array of 16-bit numbers, stored in the locations starting from a known address in 8086.	19
9.	Write an assembly language program to print Fibonacci series in 8086.	21
10.	Write an assembly language program to perform the division 15/6 using the ASCII codes. Store the ASCII codes of the result in register DX.	23

Q1.

a) Write an assembly language program to add two 16-bit numbers in 8086
(Fixed Value)

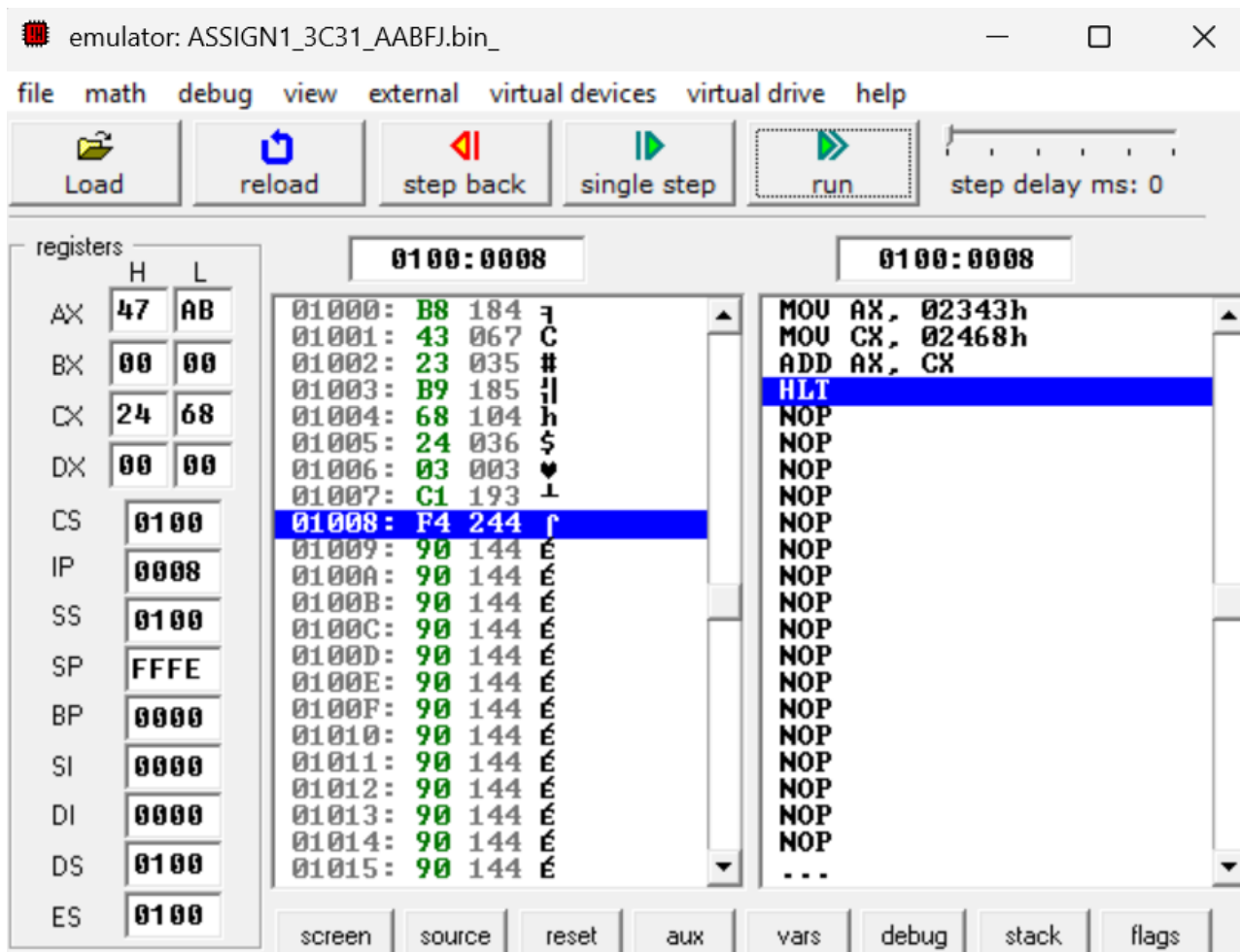
Code:

```
MOV AX, 2343H
```

```
MOV CX, 2468H
```

```
ADD AX, CX
```

```
HLT
```



b) Add two 16 - bits hexadecimal numbers (runtime values)

Code:

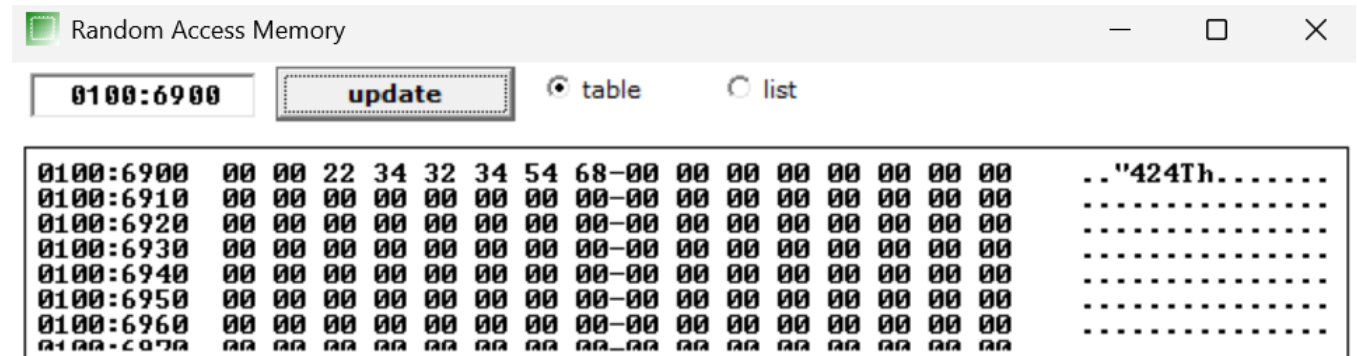
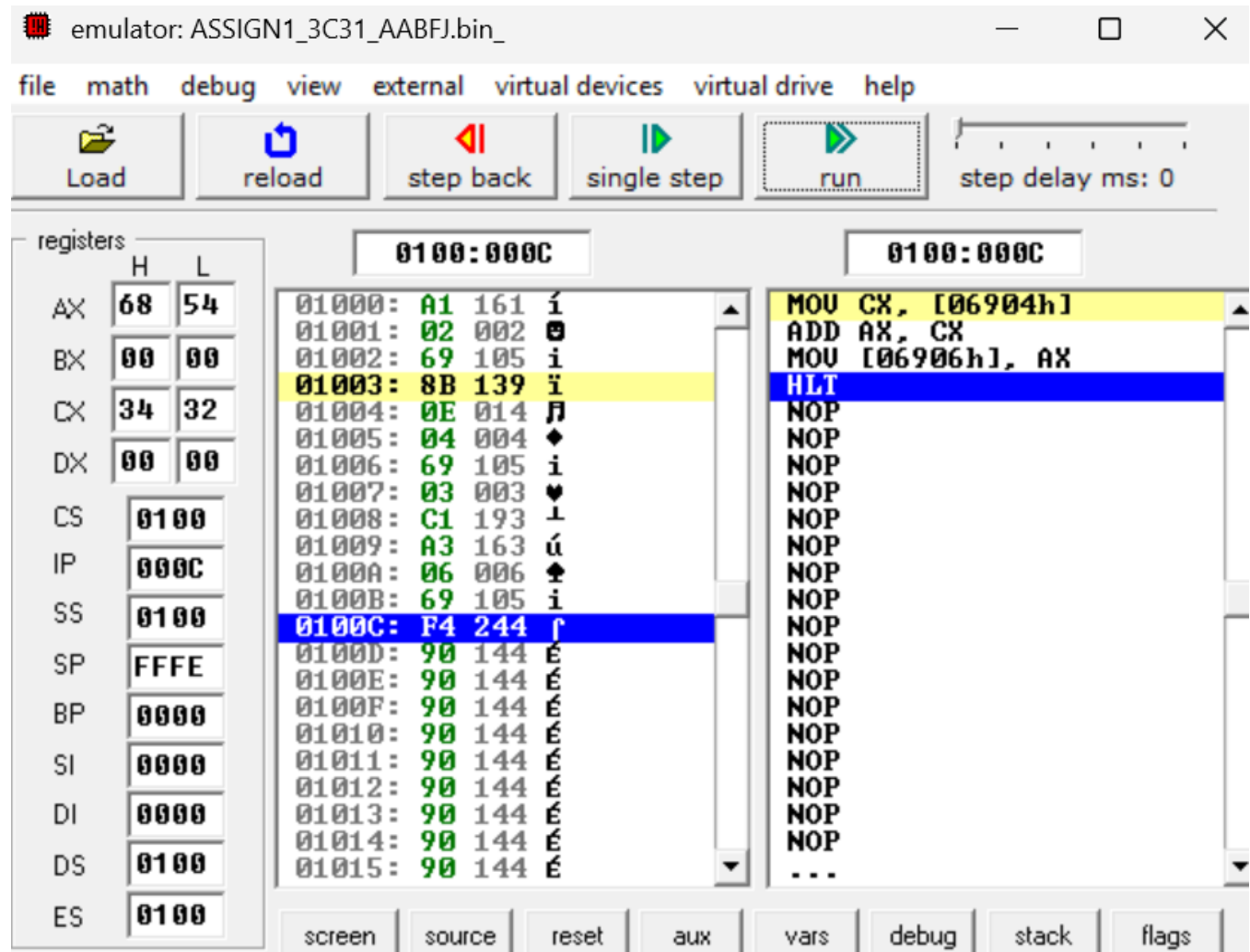
MOV AX, [6902H]

MOV CX, [6904H]

ADD AX,CX

MOV [6906H], AX

HLT



Q2.

a) Write an assembly language program to subtract two 16-bit numbers in 8086. (fixed values)

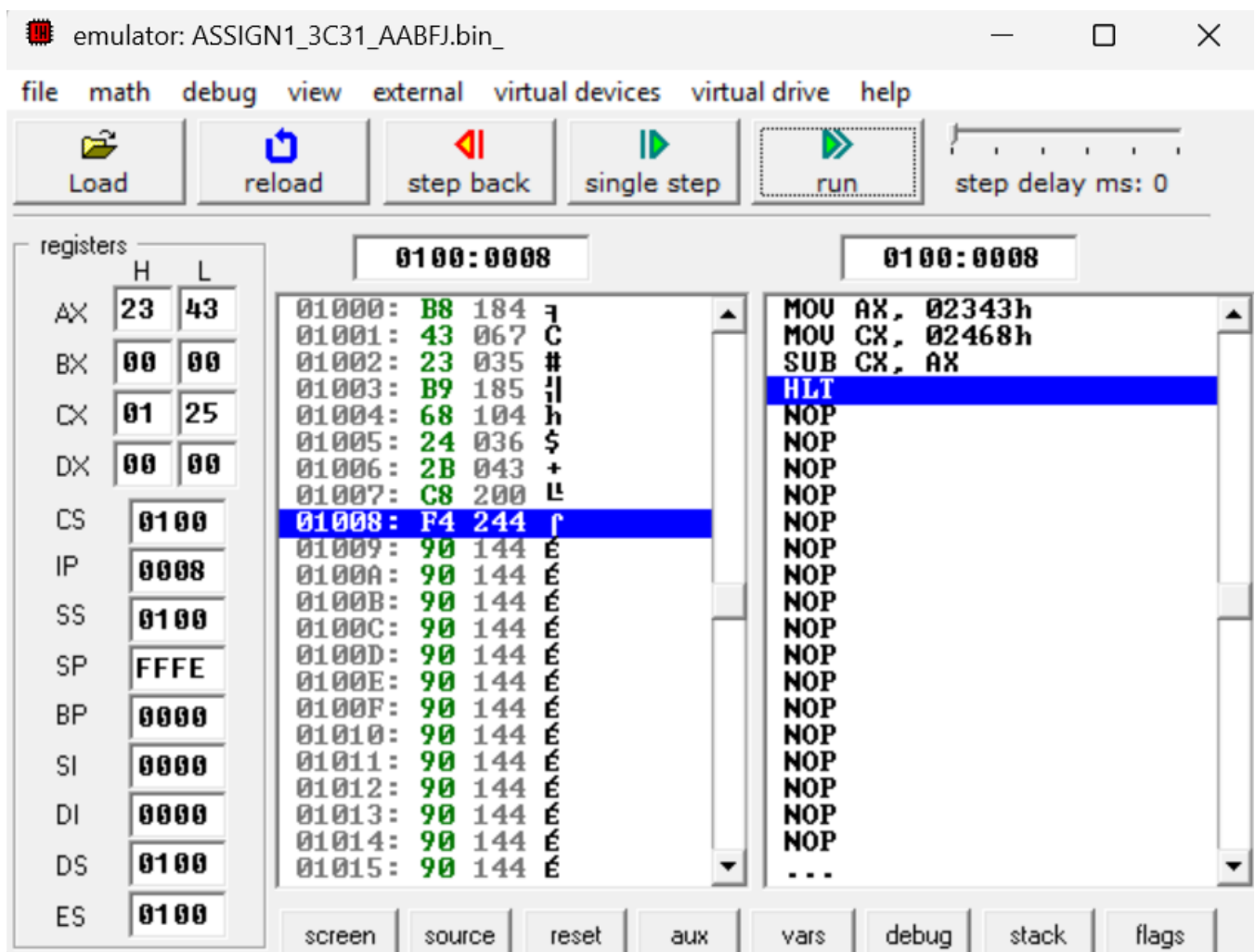
Code:

```
MOV AX, 2343H
```

```
MOV CX, 2468H
```

```
SUB CX, AX
```

```
HLT
```



b) Subtract two 16 - bits hexadecimal numbers (runtime values)

Code:

```
MOV AX, [6902H]
```

```
MOV CX, [6904H]
```

```
SUB AX,CX
```

```
MOV [6906H], AX
```

```
HLT
```

The screenshot shows an x86 emulator window titled "emulator: ASSIGN1_3C31_AABFJ.bin_". The window has a menu bar (file, math, debug, view, external, virtual devices, virtual drive, help) and a toolbar with buttons for Load, reload, step back, single step, run, and a step delay slider set to 0 ms.

On the left, the "registers" panel shows the state of various registers:

	H	L
AX	03	85
BX	00	00
CX	42	13
DX	00	00
CS	0100	
IP	000C	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

The main window is split into two panes. The left pane shows memory addresses from 01000 to 01015 with their corresponding hex values and ASCII representations. The right pane shows the assembly code being executed, with the instruction at address 0100C highlighted in blue:

```
01000: A1 161 i
01001: 02 002 0
01002: 69 105 i
01003: 8B 139 i
01004: 0E 014 n
01005: 04 004 d
01006: 69 105 i
01007: 2B 043 +
01008: C1 193 L
01009: A3 163 u
0100A: 06 006 k
0100B: 69 105 i
0100C: F4 244 r
0100D: 90 144 E
0100E: 90 144 E
0100F: 90 144 E
01010: 90 144 E
01011: 90 144 E
01012: 90 144 E
01013: 90 144 E
01014: 90 144 E
01015: 90 144 E
```

The assembly code in the right pane is:

```
MOV AX, [06902h]
MOV CX, [06904h]
SUB AX, CX
MOV [06906h], AX
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

Below the main window is a "Random Access Memory" window. It has a title bar, a close button, and a search bar. The search bar contains "0100:6900". Below the search bar is an "update" button and two radio buttons labeled "table" (selected) and "list". The main area of the RAM window displays a table of memory values:

Address	Value
0100:6900	00 00 98 45 13 42 85 03-00 00 00 00 00 00 00 00 ..ÿE!!Bà♥.....
0100:6910	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6920	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6930	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6940	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6950	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6960	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0100:6970	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00

Q3.

a) Write an assembly language program to multiply two 16-bit numbers in 8086 (fixed values)

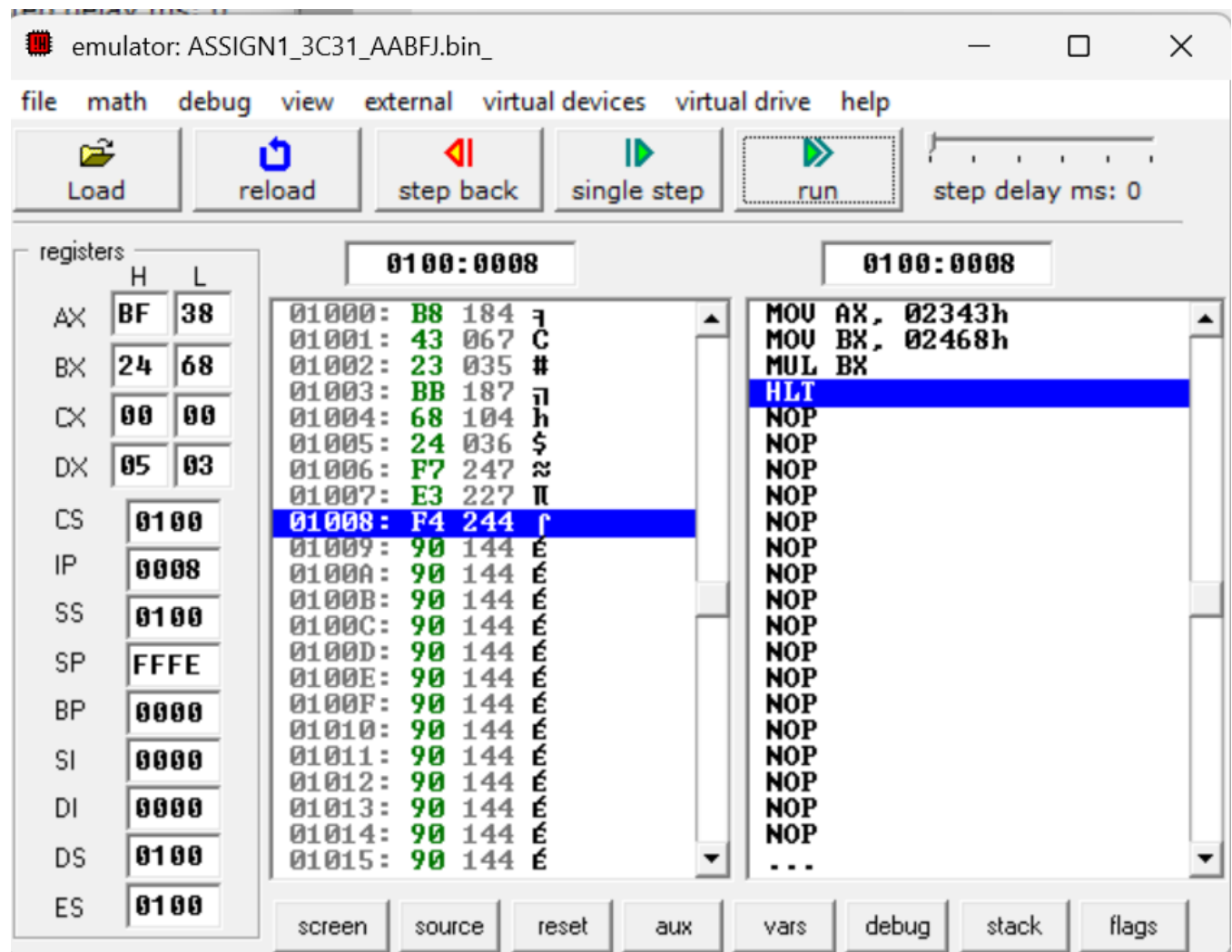
Code:

MOV AX, 2343H

MOV BX, 2468H

MUL BX

HLT



b) Multiply two 16 - bits hexadecimal numbers (runtime values)

Code:

```
MOV BX,[6902H]
```

```
MOV AX,[6904H]
```

```
MUL BX
```

```
MOV [6906H],AX
```

```
MOV [6908H],DX
```

```
HLT
```

emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	EE	0C
BX	34	12
CX	00	00
DX	18	79
CS	0100	
IP	0010	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

0100:0010

Address	Hex	ASCII
01000	8B	139 i
01001	1E	030 ▲
01002	02	002 0
01003	69	105 i
01004	A1	161 í
01005	04	004 ◆
01006	69	105 i
01007	F7	247 ≈
01008	E3	227 Π
01009	A3	163 ú
0100A	06	006 ♠
0100B	69	105 i
0100C	89	137 è
0100D	16	022 -
0100E	08	008 BACK
0100F	69	105 i
01010	F4	244 ↑
01011	90	144 É
01012	90	144 É
01013	90	144 É
01014	90	144 É
01015	90	144 É

0100:0010

```
MOV AX, [06904h]
MUL BX
MOV [06906h], AX
MOV [06908h], DX
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen source reset aux vars debug stack flags

Random Access Memory

0100:6900 update table list

Address	Hex	Decimal	Comment
0100:6900	00 00 12 34 56 78 0C EE-79 18 00 00 00 00 00 00	00 00 12 34 56 78 0C EE-79 18 00 00 00 00 00 00	..t4Ux?€y↑.....
0100:6910	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6920	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6930	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6940	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6950	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6960	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	
0100:6970	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00	

Q4.

a) Write an assembly language program to divide two 16-bit numbers in 8086. (fixed values)

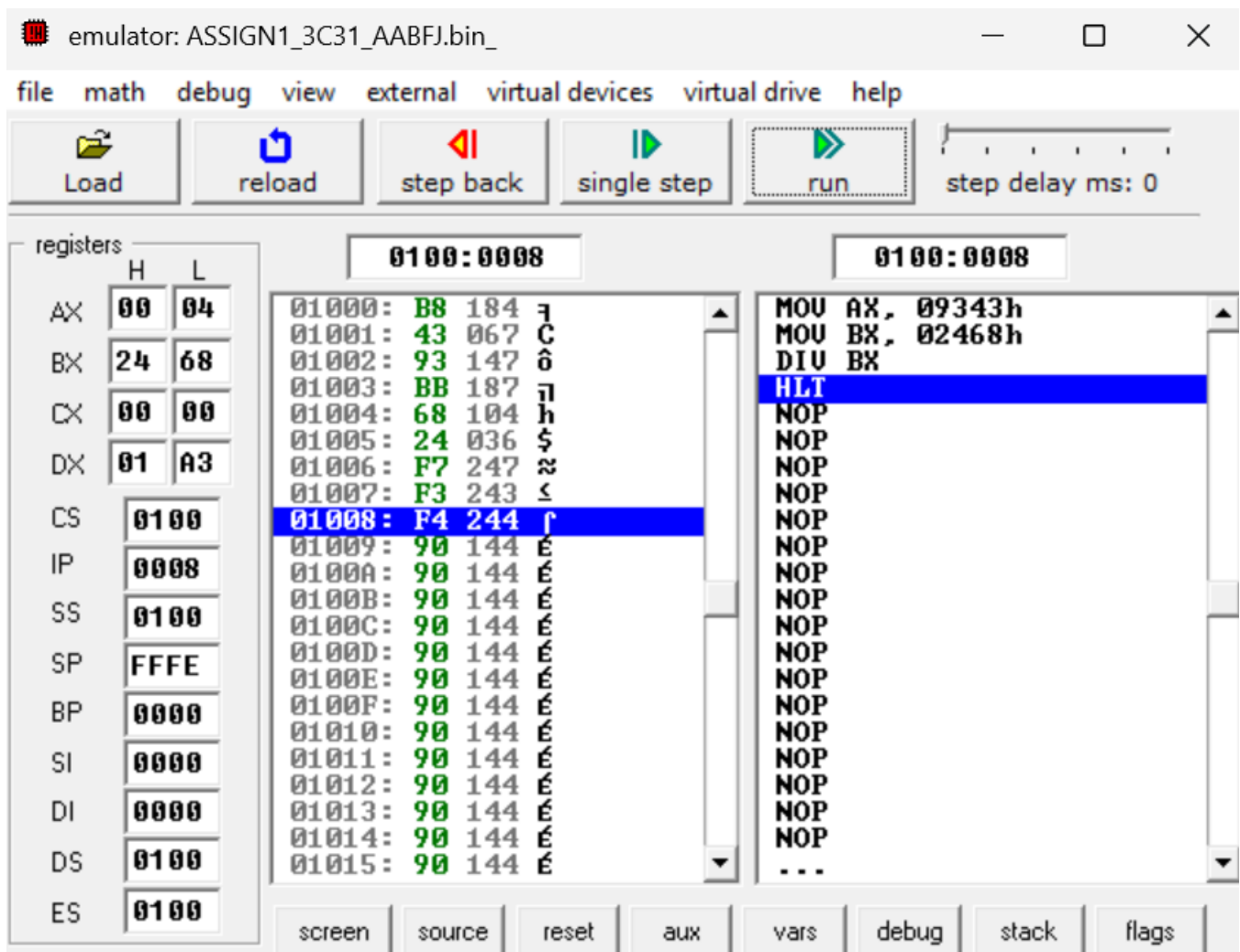
Code:

MOV AX, 9343H

MOV BX, 2468H

DIV BX

HLT



b) Divide two 16 - bits hexadecimal numbers (runtime values)

Code:

MOV BX,[6902H]

MOV AX,[6904H]

DIV BX

MOV [6906H],AX

MOV [6908H],DX

HLT

Q5.

Write an assembly language program to demonstrate AAA, AAS, AAM, AAD, DAA, and DAS in 8086.

a)AAA:

Code:

SUB AH,AH

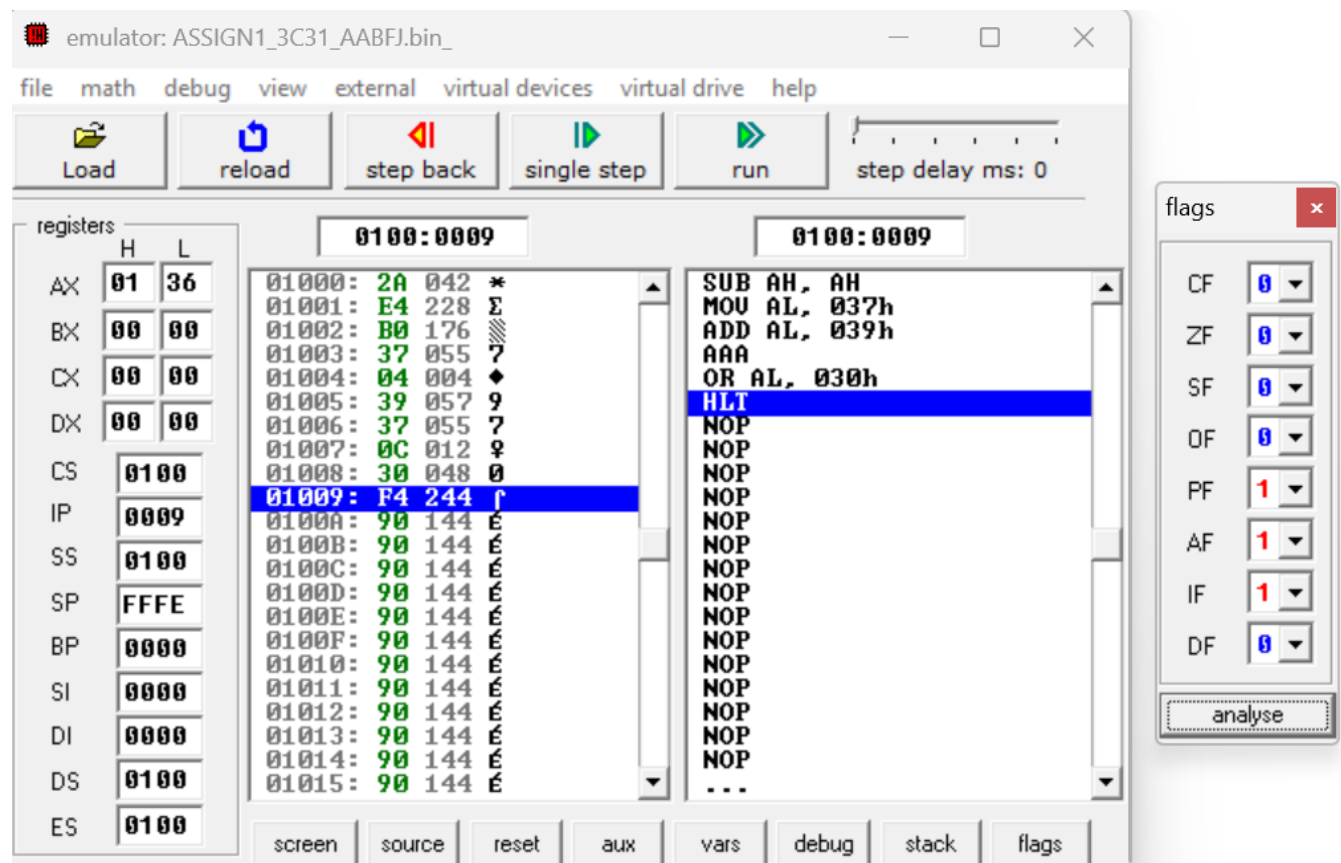
MOV AL,'7'

ADD AL,'9'

AAA

OR AL,30H

HLT



b)AAS:

Code:

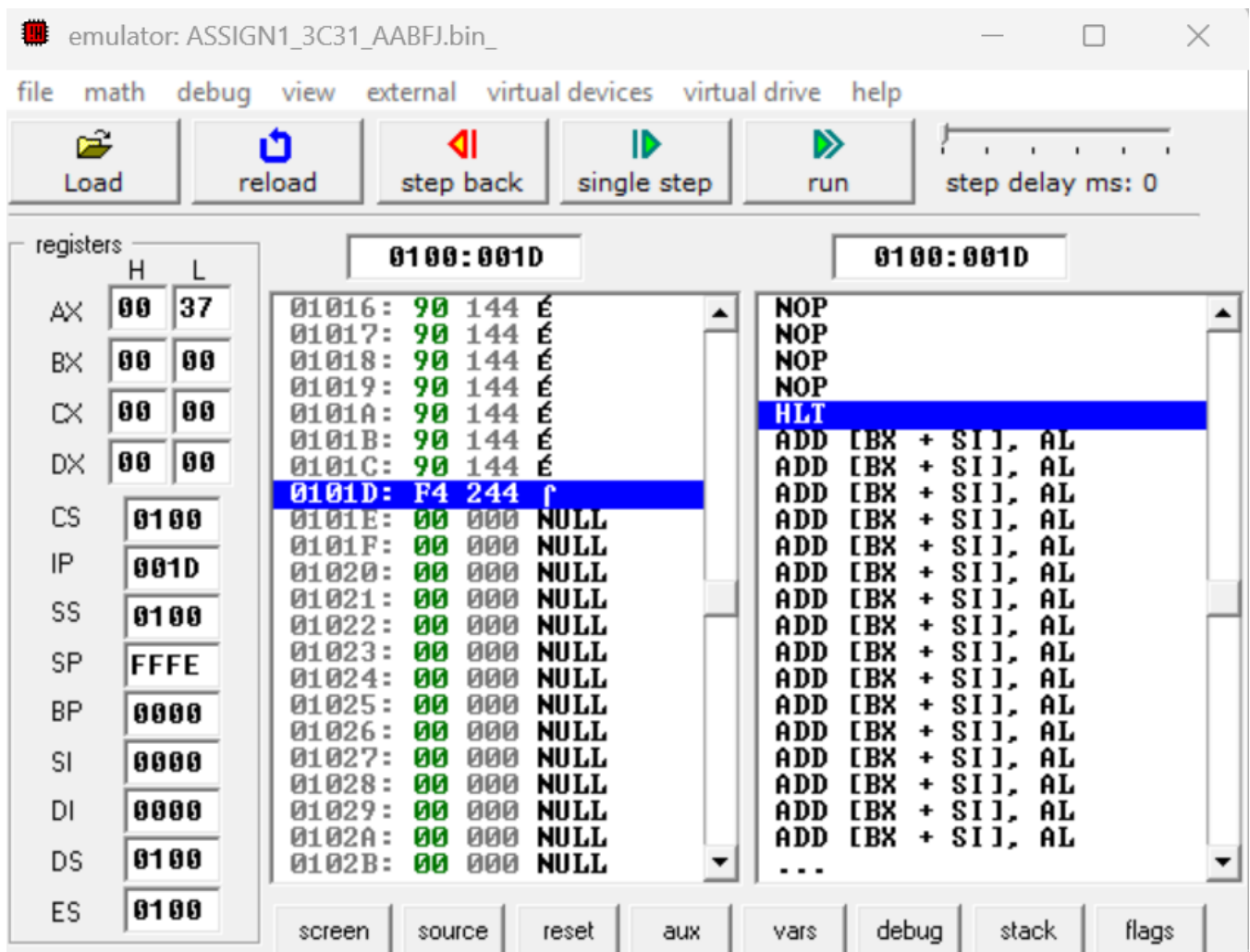
SUB AH,AH

MOV AL,'2'

ADD AL,'5'

AAS

OR AL,30H



c)AAM:

Code:

MOV AL,'7'

MOV BL,'E'

AND AL,0FH

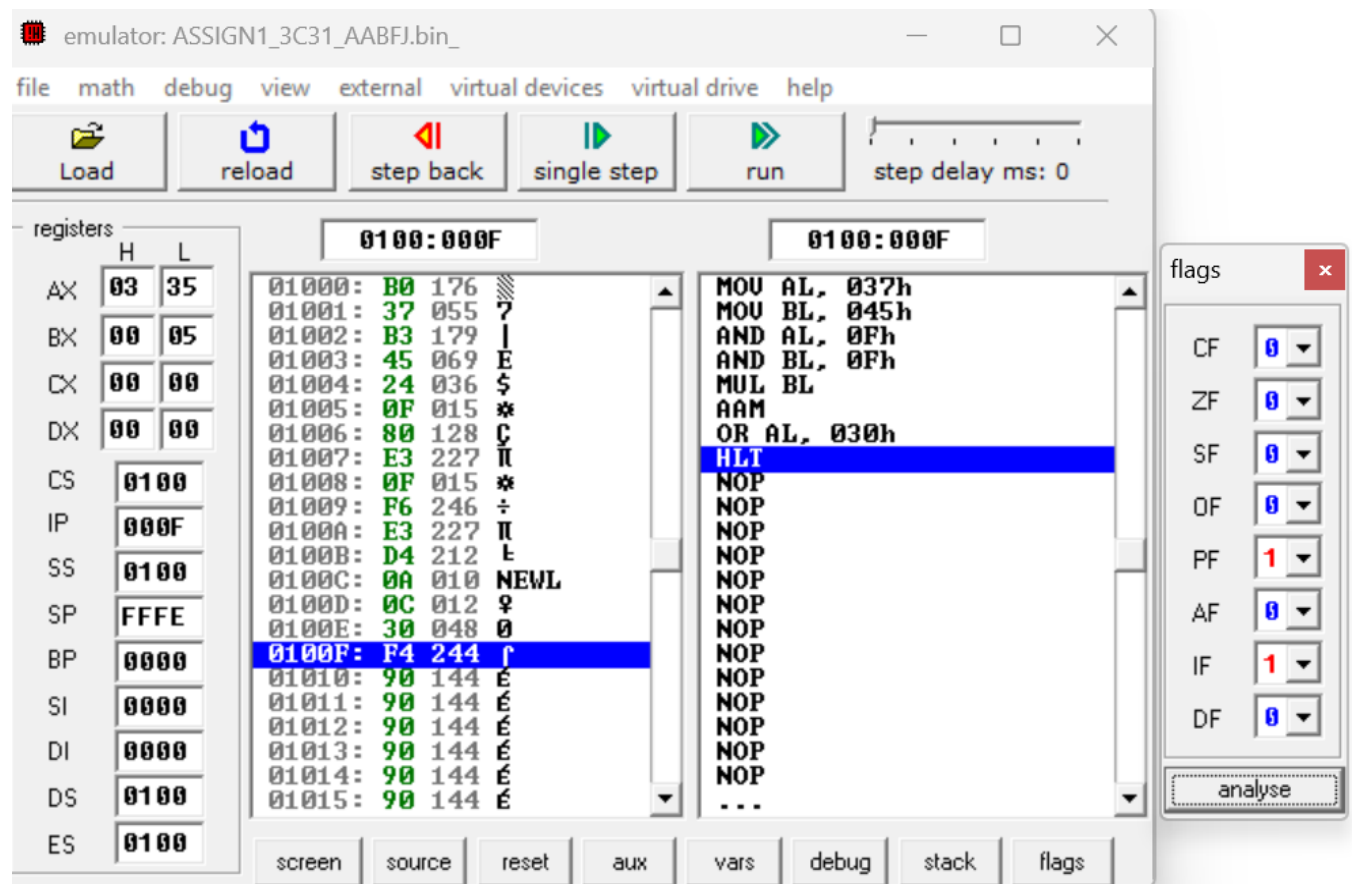
AND BL,0FH

MUL BL

AAM

OR AL, 30H

HLT



d)AAD:

Code:

MOV AX, '74'

MOV BH, '2'

SUB AX, 3030H

SUB BH, 30H

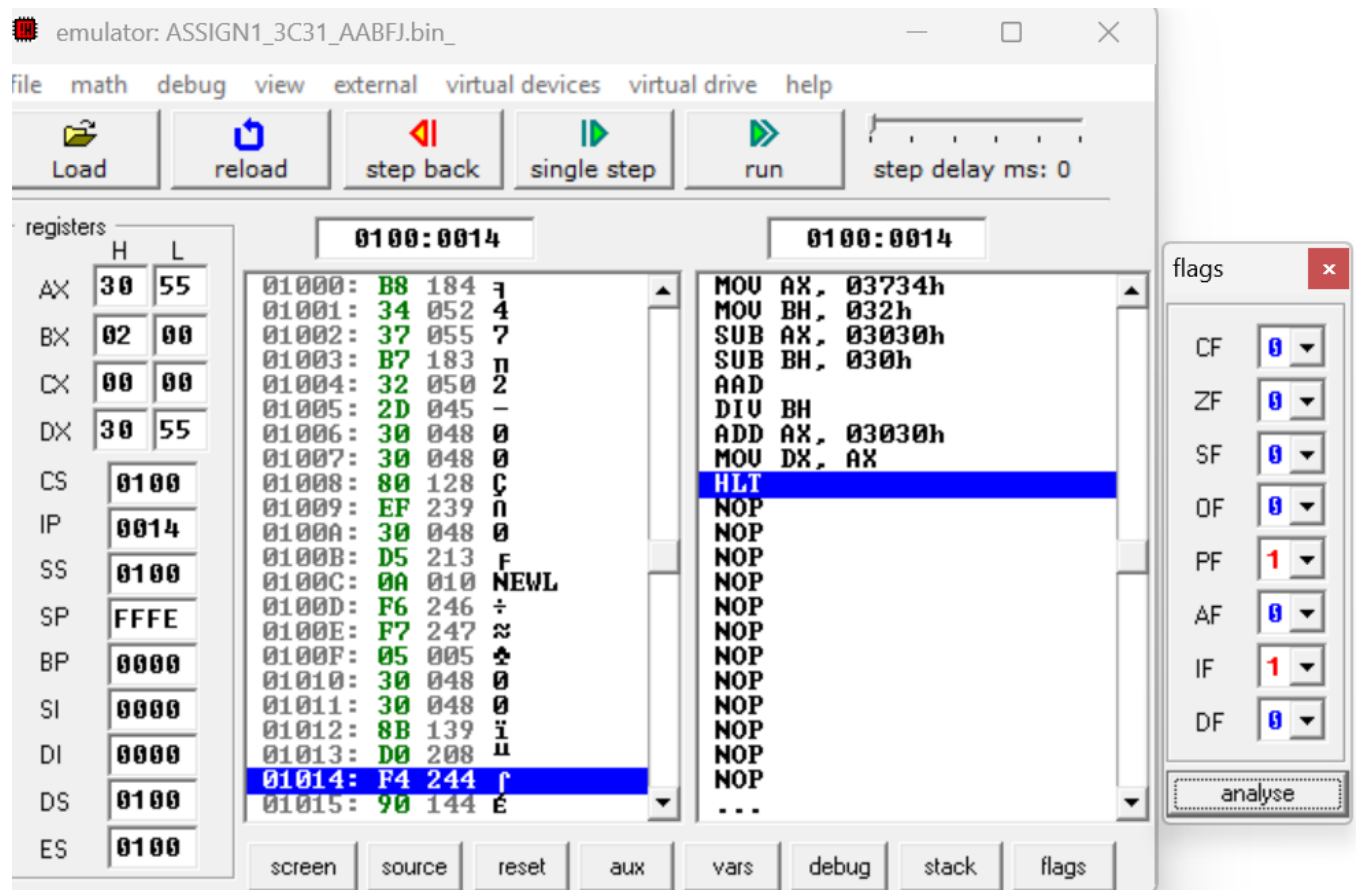
AAD

DIV BH

ADD AX, 3030H

MOV DX, AX

HLT



e)DAA:

Code:

MOV AL,91H

ADD AL,23H

DAA

HLT

emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	00	14
BX	00	00
CX	00	00
DX	00	00
CS	0100	
IP	0005	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

0100:0005

Address	Hex	ASCII
01000:	B0	176
01001:	91	145
01002:	04	004
01003:	23	035
01004:	27	039
01005:	F4	244
01006:	90	144
01007:	90	144
01008:	90	144
01009:	90	144
0100A:	90	144
0100B:	90	144
0100C:	90	144
0100D:	90	144
0100E:	90	144
0100F:	90	144
01010:	90	144
01011:	90	144
01012:	90	144
01013:	90	144
01014:	90	144
01015:	90	144

MOV AL, 091h
ADD AL, 023h
DAA
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...

0100:0005

flags

CF	1
ZF	0
SF	0
OF	0
PF	1
AF	0
IF	1
DF	0

analyse

screen source reset aux vars debug stack flags

f)DAS:

Code:

MOV AL,91H

SUB AL,23H

DAS

HLT

The screenshot shows an x86 emulator window titled "emulator: ASSIGN1_3C31_AABFJ.bin_". The interface includes a menu bar (file, math, debug, view, external, virtual devices, virtual drive, help) and a toolbar with buttons for Load, reload, step back, single step, run, and a step delay slider set to 0 ms.

On the left, the "registers" panel displays the state of various registers:

	H	L
AX	00	68
BX	00	00
CX	00	00
DX	00	00
CS	0100	
IP	0005	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

The main window is split into two panes. The left pane shows memory addresses from 01000 to 01015 with their corresponding hex values and ASCII representations. The right pane shows the assembly code being executed:

```
01000: B0 176  MOV AL, 091h
01001: 91 145  SUB AL, 023h
01002: 2C 044  DAS
01003: 23 035  HLT
01004: 2F 047  NOP
01005: F4 244  NOP
01006: 90 144  NOP
01007: 90 144  NOP
01008: 90 144  NOP
01009: 90 144  NOP
0100A: 90 144  NOP
0100B: 90 144  NOP
0100C: 90 144  NOP
0100D: 90 144  NOP
0100E: 90 144  NOP
0100F: 90 144  NOP
01010: 90 144  NOP
01011: 90 144  NOP
01012: 90 144  NOP
01013: 90 144  NOP
01014: 90 144  NOP
01015: 90 144  NOP
...
```

At the bottom, there are buttons for screen, source, reset, aux, vars, debug, stack, and flags. A "flags" window is open on the right, showing the status of various flags:

Flag	Value
CF	0
ZF	0
SF	0
OF	0
PF	0
AF	1
IF	1
DF	0

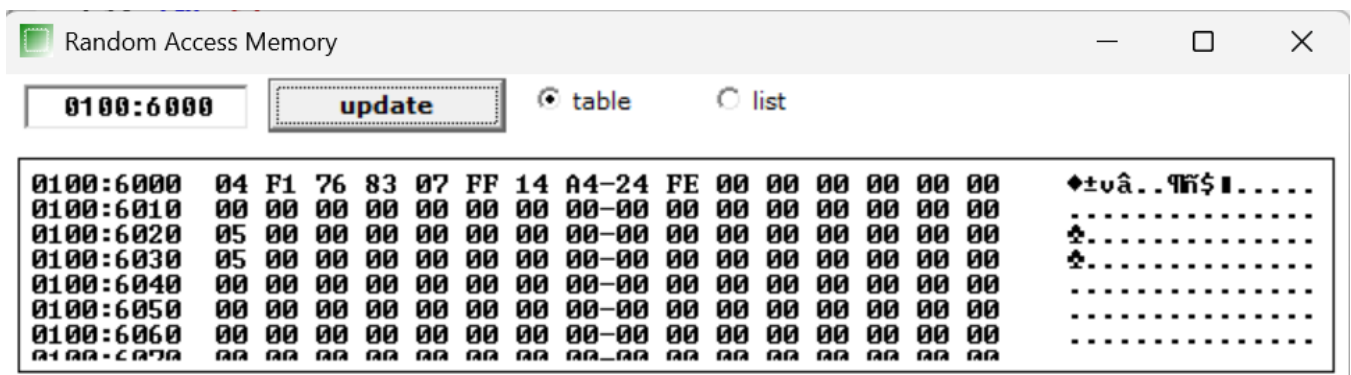
An "analyse" button is located at the bottom of the flags window.

Q6.

Write an assembly language program to find out the count of positive numbers and negative numbers from a series of signed numbers in 8086.

Code:

```
MOV CL, 0AH
MOV BL, 00H
MOV DL, 00H
LEA SI,[6000H]
L1: MOV AL,[SI]
SHL AL, 01H
JNC L2
INC DL
JMP L3
L2: INC BL
L3: INC SI
DEC CL
JNZ L1
MOV [60AAH], BL
MOV [60ABH], DL
HLT
```



emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers		0100:0022		0100:001E	
	H	L			
AX	00	FE	01022: F4 244 ↑		INC SI
BX	00	05	01023: 90 144 É		DEC CL
CX	00	00	01024: 90 144 É		JNE 0109h
DX	00	05	01025: 90 144 É		MOV [06020h], BL
CS	0100		01026: 90 144 É		MOV [06030h], DL
IP	0022		01027: 90 144 É		HLT
SS	0100		01028: 90 144 É		NOP
SP	FFFE		01029: 90 144 É		NOP
BP	0000		0102A: 90 144 É		NOP
SI	600A		0102B: 90 144 É		NOP
DI	0000		0102C: 90 144 É		NOP
DS	0100		0102D: 90 144 É		NOP
ES	0100		0102E: 90 144 É		NOP
			0102F: 90 144 É		NOP
			01030: 90 144 É		NOP
			01031: 90 144 É		NOP
			01032: 90 144 É		NOP
			01033: 90 144 É		NOP
			01034: 90 144 É		NOP
			01035: 90 144 É		NOP
			01036: 90 144 É		NOP
			01037: F4 244 ↑		...

screen source reset aux vars debug stack flags

Q7.

Write an assembly language program to convert to find out the largest number from a given unordered array of 8-bit numbers, stored in the locations starting from a known address in 8086.

Code:

```
MOV CL,0AH
```

```
LEA SI,[6000H]
```

```
MOV AL,[SI]
```

```
L1: INC SI
```

```
MOV BL,[SI]
```

```
CMP AL,BL
```

```
JC L2
```

```
JMP L3
```

```
L2: MOV AL,BL
```

```
L3: DEC CL
```

```
JNZ L1
```

```
MOV [600AH],AL
```

```
HLT
```

Address	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	ASCII
0100:6000	45	67	32	46	55	86	45	32-FA	23	FA	00	00	00	00	00	Eg2FU&E2·#·....
0100:6010	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6020	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6030	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6040	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6050	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6060	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0100:6070	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	

emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	00	FA
BX	00	00
CX	00	00
DX	00	00
CS	0100	
IP	0019	
SS	0100	
SP	FFFE	
BP	0000	
SI	600A	
DI	0000	
DS	0100	
ES	0100	

0100:0023 0100:0023

01016: A2 162 6	DEC CL
01017: 0A 010 NEWL	JNE 0107h
01018: 60 096	MOV [0600Ah], AL
01019: F4 244 J	HLT
0101A: 90 144 E	NOP
0101B: 90 144 E	NOP
0101C: 90 144 E	NOP
0101D: 90 144 E	NOP
0101E: 90 144 E	NOP
0101F: 90 144 E	NOP
01020: 90 144 E	NOP
01021: 90 144 E	NOP
01022: 90 144 E	NOP
01023: 90 144 E	NOP
01024: 90 144 E	NOP
01025: 90 144 E	NOP
01026: 90 144 E	NOP
01027: 90 144 E	NOP
01028: 90 144 E	NOP
01029: 90 144 E	NOP
0102A: 90 144 E	NOP
0102B: 90 144 E	...

screen source reset aux vars debug stack flags

Q8.

Write an assembly language program to convert to find out the largest number from a given unordered array of 16-bit numbers, stored in the locations starting from a known address in 8086.

Code:

MOV BX,6900H

MOV CL,[BX]

INC BX

MOV AX, [BX]

DEC CL

BACK: INC BX

INC BX

CMP AX,[BX]

JNC NEXT

MOV AX,[BX]

NEXT: DEC CL

JNZ BACK

MOV [6910H],AX

HLT

Random Access Memory																	
0100:6900		update		table		list											
0100:6900	12	34	24	68	32	F4	35	63-24	64	57	34	00	00	00	00	t4\$h2 r5c\$dW4...	
0100:6910	64	57	00	00	00	00	00	00-00	00	00	00	00	00	00	00	dW.....	
0100:6920	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6930	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6940	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6950	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6960	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6970	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		

emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	57	64
BX	69	23
CX	00	00
DX	00	00
CS	0100	
IP	0019	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

0100:0019

01016:	A3	163	u
01017:	10	016	►
01018:	69	105	i
01019:	F4	244	↑
0101A:	90	144	E
0101B:	90	144	E
0101C:	90	144	E
0101D:	90	144	E
0101E:	90	144	E
0101F:	90	144	E
01020:	90	144	E
01021:	90	144	E
01022:	90	144	E
01023:	90	144	E
01024:	90	144	E
01025:	90	144	E
01026:	90	144	E
01027:	90	144	E
01028:	90	144	E
01029:	90	144	E
0102A:	90	144	E
0102B:	90	144	E

0100:0019

```

MOV BX, 06900h
MOV CL, [BX]
INC BX
MOV AX, [BX]
DEC CL
INC BX
INC BX
CMP AX, [BX]
JNB 012h
MOV AX, [BX]
DEC CL
JNE 0Ah
MOV [06910h], AX
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen source reset aux vars debug stack flags

Q9.

Write an assembly language program to print Fibonacci series in 8086.

Code:

```
MOV SI, 6900H
MOV CX, 0AH
XOR AL, AL
MOV [SI], 0AH
INC SI
MOV [SI], 00H
ADD AL, 01H
INC SI
MOV [SI], AL
INC SI
MOV [SI], AL
BACK: ADD AL,[SI]
INC SI
MOV [SI], AL
DEC SI
MOV AL, [SI]
INC SI
LOOP BACK
HLT
```

Random Access Memory																— □ ×	
0100:6900		update		☒ table		☐ list											
0100:6900	0A	00	01	01	02	03	05	08-0D	15	22	37	59	90	00	00	..@Cv+..S"7YÉ.	
0100:6910	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6920	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6930	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6940	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6950	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6960	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		
0100:6970	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00		

emulator: ASSIGN1_3C31_AABFJ.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers		0100:0022		0100:0022	
	H	L			
AX	00	59	01017: 02 002 0		MOV SI, 06900h
BX	00	00	01018: 04 004 ♦		MOV CX, 0000Ah
CX	00	00	01019: 46 070 F		XOR AL, AL
DX	00	00	0101A: 88 136 ê		MOV b.[SI], 0Ah
CS	0100		0101B: 04 004 ♦		INC SI
IP	0022		0101C: 4E 078 N		MOV b.[SI], 00h
SS	0100		0101D: 8A 138 è		ADD AL, 01h
SP	FFFE		0101E: 04 004 ♦		INC SI
BP	0000		0101F: 46 070 F		MOV [SI], AL
SI	6900		01020: E2 226 Γ		INC SI
DI	0000		01021: F5 245 J		MOV [SI], AL
DS	0100		01022: F4 244 ↑		ADD AL, [SI]
ES	0100		01023: 90 144 É		INC SI
			01024: 90 144 É		MOV [SI], AL
			01025: 90 144 É		DEC SI
			01026: 90 144 É		MOV AL, [SI]
			01027: 90 144 É		INC SI
			01028: 90 144 É		LOOP 017h
			01029: 90 144 É		HLT
			0102A: 90 144 É		NOP
			0102B: 90 144 É		NOP
			0102C: 90 144 É		...

screen source reset aux vars debug stack flags

Q10.

Write an assembly language program to perform the division 15/6 using the ASCII codes. Store the ASCII codes of the result in register DX.

Code:

```
MOV AH,'1'  
MOV AL,'5'  
MOV BH,'6'  
SUB AX,3030h  
SUB BH,30H  
AAD  
DIV BH  
ADD AX,3030H  
MOV DX,AX  
HLT
```

